

Using ID3 Decision Trees to Classify and Analyze Data

Jacob A. Geldbach
College of Engineering: CS
University of Idaho
Moscow Idaho, United States
geld8788@vandals.uidaho.edu

Abstract—In this project, Information Theory was used in order to build an ID3 decision tree from a subset of data entries, this subset known as the training subset was picked randomly from a given set of 100 example data entries. Each entry is defined as 7 different binary attributes indicating if that attribute for that example was true or not. The outcome attribute of each entry indicates if the robot in that scenario asked for help or not or decides on its own. The remainder of data entries not included in the training subset then are used to make up the testing subset. The ID3 Decision tree built from the training subset was then tested against the testing subset where accuracy and other error measuring metrics were recorded and compared against runs of different training subset size.

Index Terms—Decision Tree, ID3, Information Theory, Entropy, Supervised Learning, Classification

I. ALGORITHM DESCRIPTION

The algorithm for this project was aimed at producing a decision tree from a subset of data so that the testing subset's error rates could be classified and compared. Specifically a Ross Quinlan 1986 ID3 greedy most information gained first implementation was used to generate a decision tree. Wherein the program will begin with a training subset size input from 1-99, wherein that many rows of data from the given 100 row dataset are randomly chosen to comprise that run or trials training subset. Once the training subset for this run has been made up, each attributes Information Gain is calculated using the entropy equation shown in Eq. (1), which is also talked about further in the next section. The attribute with the highest calculated Information Gain is then chosen to be the root of the tree, and this is repeated until there are no attributes with non-zero information gain left from this training subset. Each row that comprises the testing subset (100 row dataset - the training subset) is then tested against this built tree, where starting at the root of the tree, the attribute is looked up in the datarow and the tree is traversed until a terminal yes or no state is seen indicating that the decision tree has decided if the robot on that run asked for help or not. The answer the tree found for that data row is recorded. Once all data rows in the testing subset are ran through the decision tree, a confusion matrix for this trial is calculated by comparing the decision trees answer (predicted) vs the data rows answer (actual). This confusion matrix allows for accuracy, recall and precision to be calculated for that trial. The program was built so that many trials at this size could be ran (all with same sized training sets

but randomly made up) and the error metrics could then be averaged across the multiple runs of the same training subset size.

Measuring Information

The entropy calculation shown in Eq. (1) is what the algorithm uses each iteration to choose the next attribute in the tree. Entropy measures how mixed the Yes/No decisions are in a set of rows, and the Information Gain of a candidate attribute is the entropy of the rows at that node minus the size-weighted average of the entropies of the two subsets that splitting on that attribute produces. Omitting any attributes that have already been chosen that run from the calculation, it simply chooses the attribute with the highest Information Gain, meaning the attribute whose split leaves the least remaining entropy, based on the attributes and their probabilities in the comprised training subset.

$$I(p(v_1), \dots, p(v_n)) = - \sum_i p(v_i) \log_2 p(v_i) \quad (1)$$

Handling Undefined Cases

There were two undefined scenarios in the information calculating and building of the tree that needed to be addressed. The first was when a class has $p_i = 0$. This will lead to Eq. (1) being undefined as $\log_2(0)$ goes to $-\infty$, also when $p_i = 0$ that term has a probability of 0 so that single term is simply skipped and not included in the entropy sum. The 2nd case being when two attributes produce valid but equal gain. The tie breaker used in this implementation was simply which attribute came first in the CSV ordering, and this was kept consistent across all runs since the CSV was structured the same throughout.

Measuring Performance

Performance for a specific trials decision tree was measured by calculating the accuracy, precision and recall which was derived from that given runs confusion matrix results. Each training set size was ran on 50 different trials, each with its own randomly comprised same size training subset. The error metrics were then averaged across all the trials.

TABLE I
ID3 DECISION TREE — SWEEP RESULTS (50 TRIALS PER TRAINING-SET SIZE)

Train Size	Trials	Train Acc.	Test Acc.	Test Prec.	Test Recall	Mean Tree Size	Mean Tree Depth	Most Common Root Attr.
10	50	1.000	0.697	0.759	0.751	6.96	2.90	Safe Situation
20	50	1.000	0.816	0.878	0.820	11.12	3.88	Safe Situation
30	50	1.000	0.829	0.870	0.852	15.00	4.44	Safe Situation
40	50	1.000	0.878	0.897	0.910	17.96	4.74	Safe Situation
50	50	1.000	0.892	0.910	0.918	19.72	4.96	Safe Situation
60	50	1.000	0.924	0.928	0.951	21.52	5.22	Safe Situation
70	50	1.000	0.949	0.954	0.964	22.28	5.12	Safe Situation
80	50	1.000	0.952	0.949	0.974	23.12	5.24	Safe Situation
90	50	1.000	0.970	0.964	0.993	23.12	5.14	Safe Situation

II. RESULTS

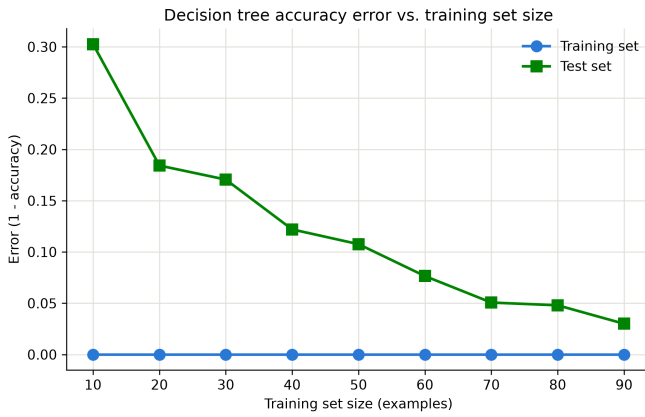


Fig. 1. Accuracy error (1–accuracy) vs. training set size, training-set curve vs. held-out test-set curve (mean over 50 trials per size).

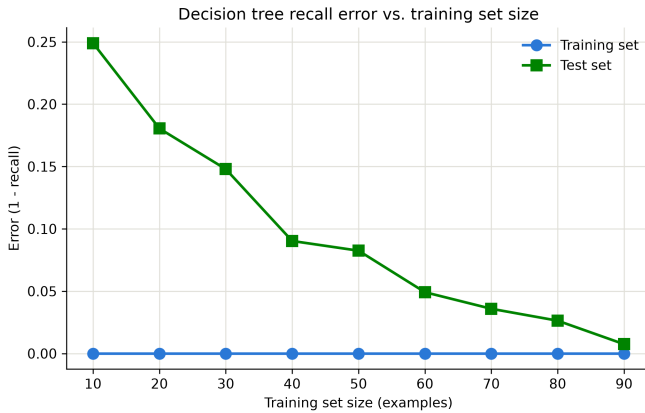


Fig. 2. Recall error (1–recall) vs. training set size, training-set curve vs. held-out test-set curve (mean over 50 trials per size).

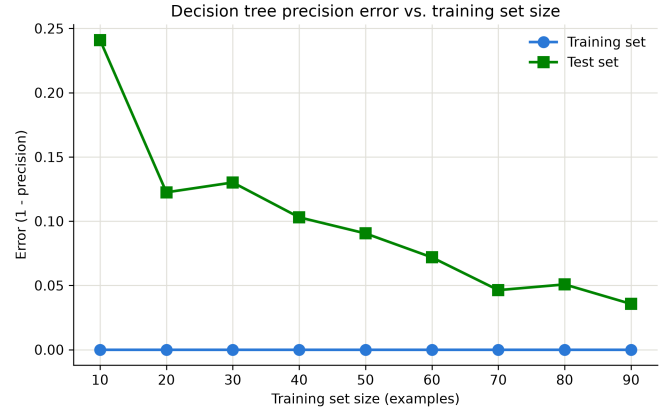


Fig. 3. Precision error (1–precision) vs. training set size, training-set curve vs. held-out test-set curve (mean over 50 trials per size).

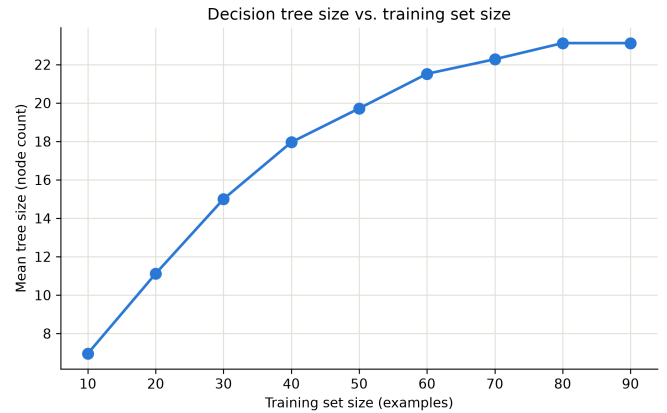


Fig. 4. Mean decision tree size (node count) vs. training set size (mean over 50 trials per size).

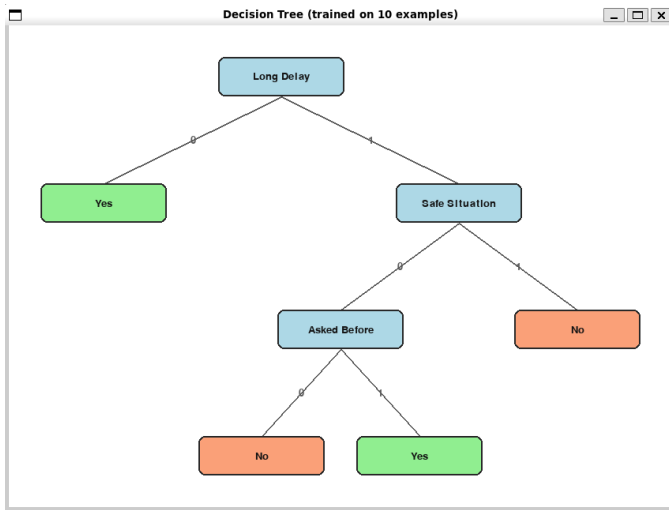


Fig. 5. Decision tree built from a training set of 10 examples.

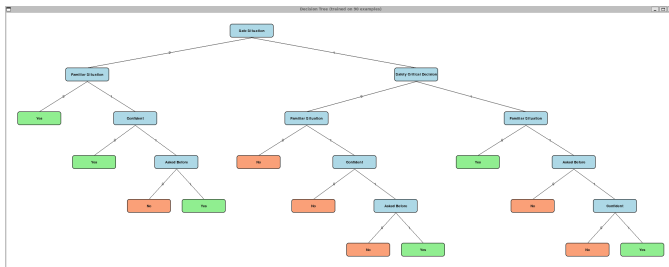


Fig. 6. Decision tree built from a training set of 90 examples.

III. DISCUSSION / CONCLUSIONS

The results clearly show that the larger the training subset, the more accurate the result of the decision tree was when tested against the testing subset across all three metrics. Also the tree size scaled significantly with the size of the training subset and the root attribute was Safe Situation in every trial at every size. Intuitively this seems like appropriate programming for a robot in regards to when it needs to ask for help or not.

The decision tree when ran against its own training subset it was built from had an expected 100 percent success rate so the overfitting standard trend is a bit harder to see when looking at the testing subsets trend curve, but with the addition of more data rows the trending closer to the perfect training subset does seem to start to plateau. This seems to be due to enough attributes becoming non-zero in the training subset therefore the decision trees number of nodes begin to grow no matter how little an attribute gives in regards to information. Although there is not a size in training subset where we explicitly see the testing subset start to do worse, this may be due to the limited size of the testing subset, where as the training subset grows the testing subset size shrinks and perhaps if we had more data to comprise the testing subset as the training subset and the decision tree it built grew less efficiently, the classic overfitting curve trend would be seen. To the point where between training subset sizes of 70 and 80 we

see the precision metric actually get worse, where precision measures what fraction of the situations where the decision tree called for help genuinely required it, essentially the tree began producing more false positives at that point. Although asking for help in a situation where the robot did not have to is the safer direction to err in as opposed to false negatives.

The strengths of the ID3 greedy first information based building of a decision tree is that it is a relatively simple math calculation that can be iterated over easily by computers in order to build out a pretty effective tree for coming to a conclusion. The greedy first approach is also a strength as this local focused way of choosing the next best attribute without taking into account the already built tree makes much larger training datasets possible to be used. Then the solution itself would be more accurate with the size of the training set being larger as we were able to see in the data collected above.

USE OF AI

Coding

The Claude Sonnet 5.0 Model was used for the initial implementation of the project's argument parsing, the CSV data loading and train/test sampling, the per-trial results logging, the matplotlib graph generation in analyze.py, and the Pygame set up used to render a built decision tree. This was done by providing the model with detailed task level .md files, one per component, kept in the project's tasks/ directory. Other than that, the core ID3 logic was written by hand, specifically the entropy and information gain calculations and the recursive tree building that applies them.

Writeup

The Claude Opus 5 Model was used for all the Latex structural code and library inclusions needed for this document, including the formatting of the results table and the figure blocks holding the graphs and the decision tree screenshots. The model was also used for grammar and spelling checks of the section content, all of the content in the sections are my own words. All of the data in the tables and graphs were gathered and added to the document by manually running the decision tree program.